Day 1 – Environment & Project Setup

🏷️ Teacher Prep & Classroom Setup

1. **Pre-class Checklist**:
   - Ensure you have Python 3.12+ installed.
   - Open a clean terminal window in an empty project directory.
   - Have a web browser open and ready to navigate to `http://127.0.0.1:8000/`.
2. **Prerequisites Checklist**:
   - Students should have standard terminal environments working on their OS (Mac/Linux or Windows Git Bash/PowerShell).
3. **Required Material**:
   - Whiteboard or slide showing the end-to-end system architecture (User → Orders API → DB/GameKey → Expiration Engine → Celery → Webhook Queue → Publisher).

🎯 Learning Objectives

By the end of this class, students will be able to:

- Install and configure modern Python package management using `uv`.
- Initialize and activate a virtual environment.
- Scaffold a new Django project and custom app.
- Securely configure environment variables using `python-decouple`.
- Initialize a Git repository and prevent secrets leaks using `.gitignore`.
- Run the Django development server and verify the default setup.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus / Teacher Action
1. Hook & Big Picture	15 min	Interactive overview of the bootcamp outcome (game key expiry & publisher webhooks).
2. Key Concepts & Core Ideas	15 min	Introduction to MTV, API backends, package management (`uv`), and configuration security.

3. Live Coding Walkthrough	45 min	Live setup of python environment, Django, and decoupling configuration.
4. Check for Understanding	10 min	Q&A on virtual environments, secrets management, and DEBUG mode.
5. Class Wrap-up & Checkpoint	5 min	Verification of students' local environments.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (15 min)

- **Teacher Action**: Open a diagram showing the complete workflow. Explain the business problem: digital game key marketplaces need to deliver keys to users, track their expirations, and automatically notify publishers when a key expires so they can deactivate it on their servers.
- **Big Picture**: Show the finished flow: User buys key → key expires → Celery fires webhook → publisher receives it. Explain to students: *Make them care about the outcome before writing a single line.*
- **Instructor Q&A**:
  - **Question**: Why do we need webhooks instead of polling?
    - *Answer*: Polling is inefficient because clients have to repeatedly query the database (creating high traffic and latency). Webhooks are event-driven: our system pushes notifications only when an event happens, reducing server load and ensuring near-instant updates.
  - **Question**: Why is this a production-grade problem?
    - *Answer*: It involves cross-system communication, payload security (signing), concurrency (race conditions), reliability (retries, queues), and audit logging.

2. Key Concepts & Core Ideas (15 min)

Introduce the following definitions to the students:

- **What is Django?** MTV (Model-Template-View) framework. Mention that we'll use it purely as a REST API backend with no HTML templates.
- **What is DRF?** Django REST Framework. A toolkit built on top of Django that gives us serializers, viewsets, and routers for building RESTful APIs.
- **Virtual environments**: Why isolation matters (preventing dependency conflicts and ensuring reproducibility across developers' machines).
  - *Instructor Note - Why do we isolate?* Without virtual environments, installing packages globally can overwrite dependencies needed by other projects on the same machine, causing conflicts and breaking applications.

- **DEBUG=True vs False**: Django displays highly detailed error pages containing traceback information. Explain that this is crucial for development but a critical security leak in production.
  - *Instructor Note - Why is it a leak?* It exposes database queries, settings, system file paths, configurations, and environment variables to the end user.
- **Why python-decouple?** Reading from `os.environ.get()` is functional but `decouple` simplifies casting types (e.g. converting a string `"True"` to boolean `True`) and handling defaults and local `.env` files in a clean API.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Tooling Installation & Scaffolding

- **Explain**: Contrast standard `pip` + `venv` with `uv`. Why are we using `uv`? Speed, lockfiles, and reproducibility. Run the installation and scaffolding commands while students follow along.
- **Code**:

```
# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# Create and activate virtual environment
uv venv
source .venv/bin/activate

# Add dependencies
uv add django djangorestframework python-decouple celery redis pytest-django

# Scaffold project and app
django-admin startproject gamekey_platform .
python manage.py startapp games
```

- **Verify**: Confirm that every student sees `(.venv)` in their command line prompt. Pause here to ensure everyone is inside the active virtual environment before moving on.
- ⚠ **Common Pitfalls**:
  - **Forgetting to activate the venv**: If students skip `source .venv/bin/activate`, they will install dependencies globally or get "django not found". Teach them to run `which python` to verify the path points to `.venv`.
  - **Windows path differences**: Flag that on Windows, the activation command is `.venv\Scripts\activate` rather than `.venv/bin/activate`.

Step 3.2: Environment Variable Configuration

- **Explain**: Explain why secrets (such as the Django `SECRET_KEY`) should never go into version control (source code). Show what happens if you push a secret key to a public GitHub repository. Create the local environment file.

- **Code**: Create in the project root directory:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
```

- **Verify**: Check that the `.env` file is created and contains the correct key-value pairs.

Step 3.3: Django settings.py Decoupling

- **Explain**: Now we need to modify Django's settings to read our newly created `.env` file rather than having hardcoded variables.
- **Code**: Update `gamekey_platform/settings.py` to import and use `python-decouple`:

```
from decouple import config

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
```

- **Verify**: Run the development server to verify the configuration is loaded correctly:

```
# Verify setup
python manage.py runserver
```

  Navigate to `http://localhost:8000/` and verify the Django welcome page loads.

- ⚠ **Common Pitfalls**:
  - ○ **`SECRET_KEY` still hardcoded**: Students might add `python-decouple` but forget to update the actual variables in `settings.py`. Test this by removing the key from `.env` and seeing if Django throws an error on startup.

Step 3.4: Git Initialization

- **Explain**: Set up version control. Introduce the `.gitignore` file and emphasize that the `.env` file must be ignored immediately to prevent leaking the secret keys.
- **Code**:

```
git init
```

  Create `.gitignore` and include `.env` (along with standard Python/Django patterns: `__pycache__`, `.venv`, `*.pyc`, etc.):

```
.env
.venv/
*.pyc
__pycache__/
db.sqlite3
```